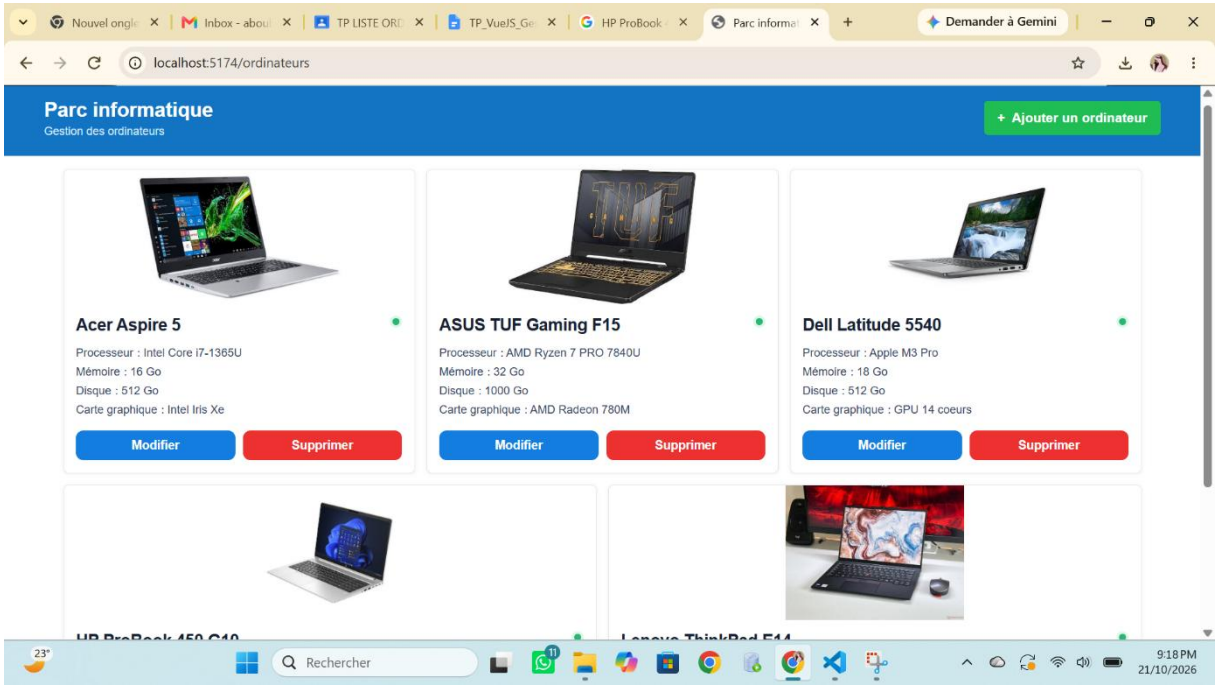
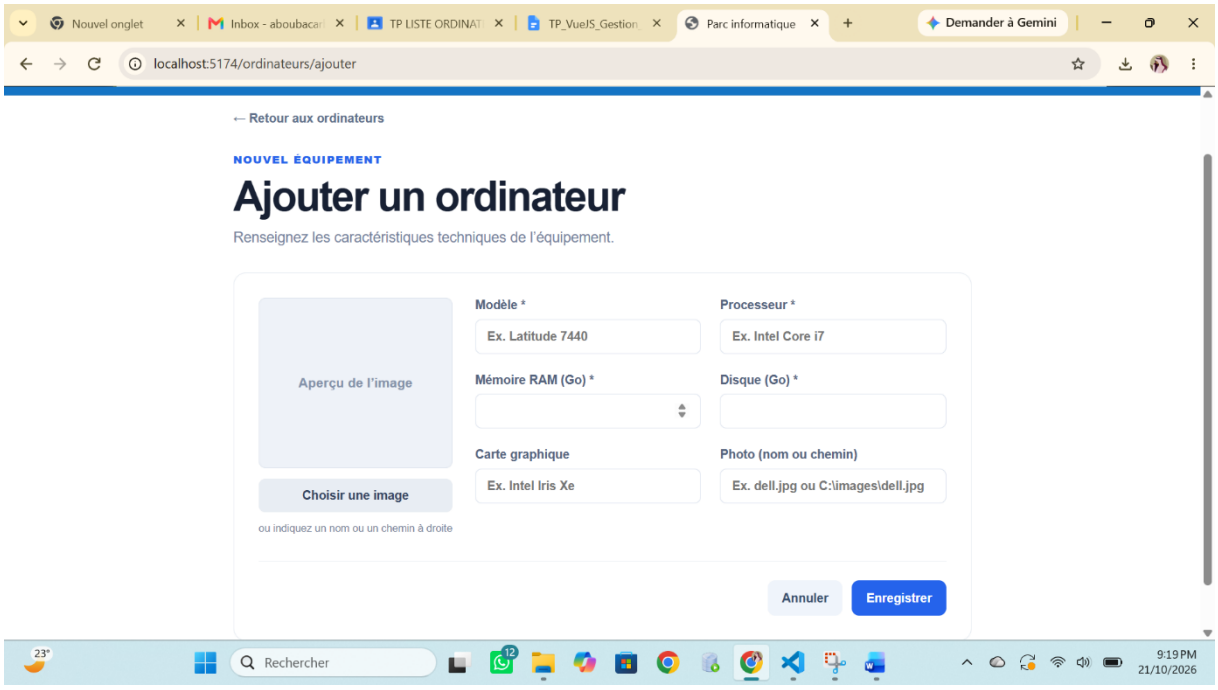
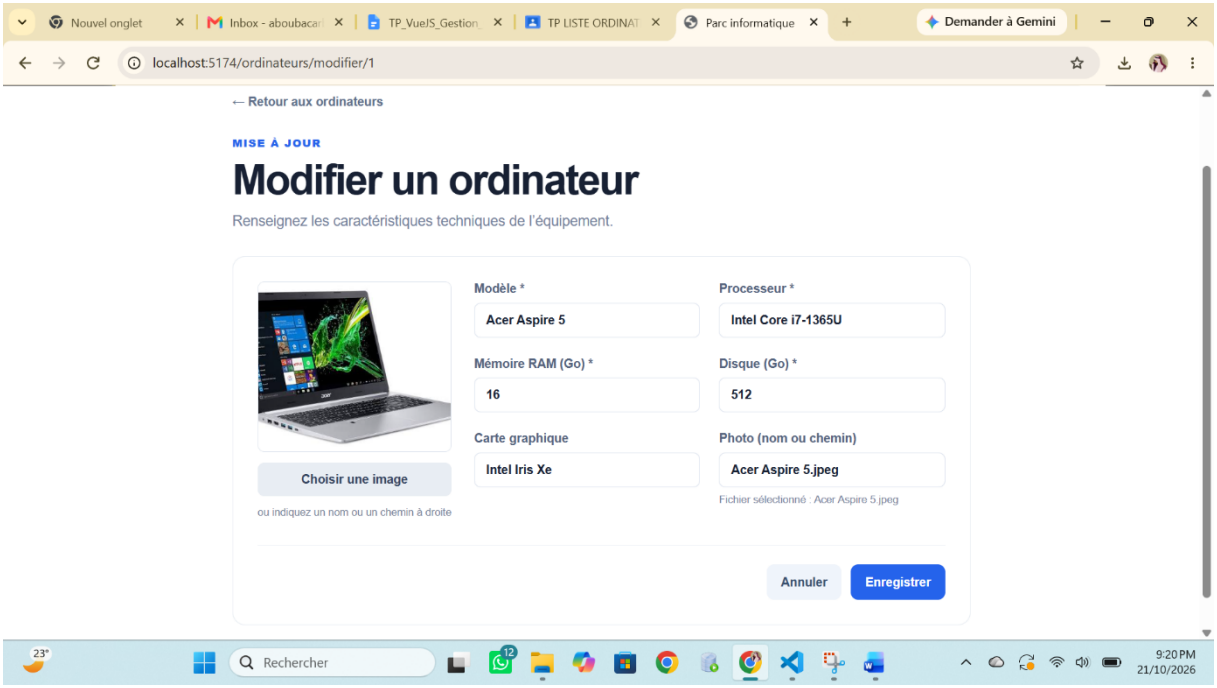
Ecran d'Accueil



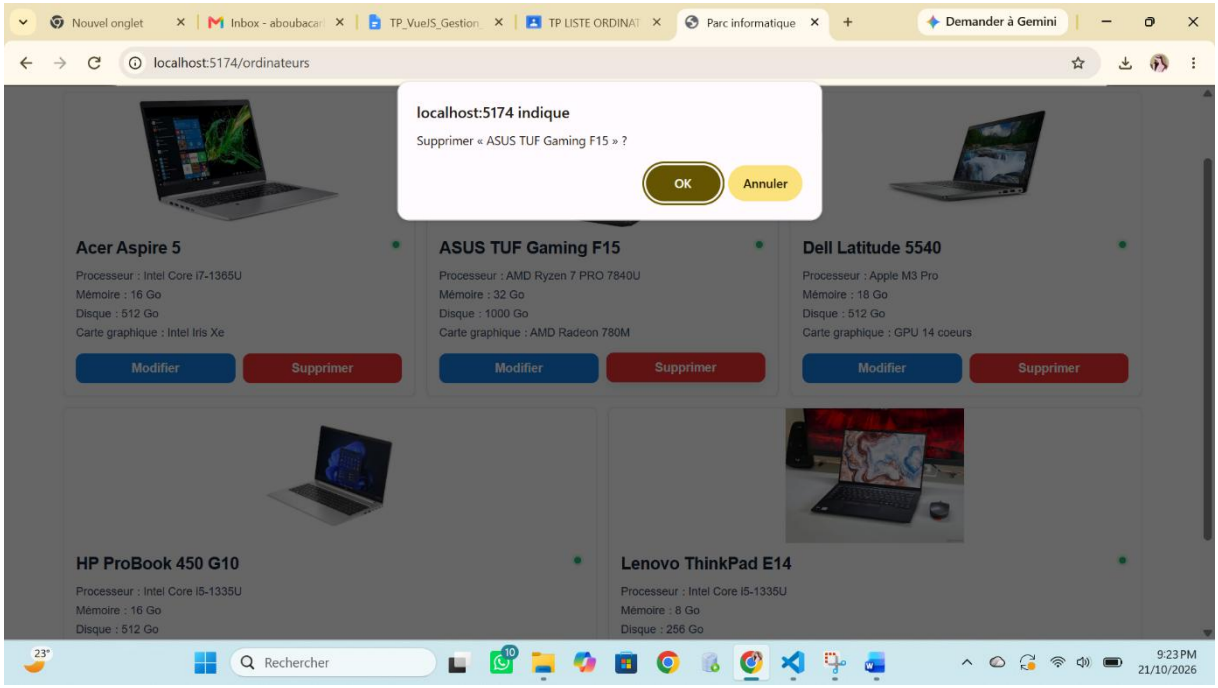
L'interface Ajouter un Ordinateur



L’interface Modifier un Ordinateur



La Confirmation de Vouloir Supprimer Un Ordinateur



Compte rendu — Gestion d'un parc informatique

1. Présentation du projet

Ce projet consiste à réaliser une application web avec Vue.js 3 permettant de gérer un parc d'ordinateurs. L'application fonctionne entièrement côté client, sans serveur ni base de données externe.

Elle permet de consulter les équipements enregistrés, d'en ajouter de nouveaux, de modifier leurs caractéristiques et de les supprimer.

2. Technologies utilisées

- Vue.js 3 ;
- Vite pour la création et la compilation du projet ;
- Vue Router pour la navigation entre les pages ;
- JavaScript avec la Composition API ;
- HTML et CSS pour la structure et la mise en forme ;
- `localStorage` pour conserver les données dans le navigateur.

3. Organisation du projet

```text
src/
├── assets/
│   └── images/
├── components/
│   └── OrdinateurCard.vue
├── data/
│   └── ordinateurs.js
├── services/
│   └── storage.js
├── views/
│   ├── ListeOrdinateurs.vue
│   └── EditOrdinateur.vue
├── App.vue
├── main.js
└── style.css
```

4. Composants et fichiers principaux

`App.vue`

Ce composant contient la structure générale de l'application :

- la barre supérieure ;
- le titre « Parc informatique » ;
- le sous-titre « Gestion des ordinateurs » ;
- le bouton d'accès à l'ajout d'un ordinateur ;
- l'emplacement du contenu fourni par Vue Router avec `<RouterView />`.

`OrdinateurCard.vue`

Ce composant réutilisable affiche un ordinateur sous forme de carte. Il présente :

- la photo de l'ordinateur ;
- le modèle ;
- le processeur ;
- la mémoire RAM ;
- la capacité du disque ;
- la carte graphique ;
- les boutons « Modifier » et « Supprimer ».

La carte reçoit les informations de l'ordinateur grâce aux `props` et signale
la demande de suppression au composant parent avec un événement.

`ListeOrdinateurs.vue`

Cette vue affiche la liste des ordinateurs avec une boucle `v-for`. Elle gère :

- l'affichage des cartes ;
- la recherche par modèle ou processeur ;
- le tri par modèle, mémoire ou disque ;
- la suppression après confirmation ;
- la réinitialisation des exemples ;
- l'adaptation de la grille selon le nombre de cartes.

`EditOrdinateur.vue`

Cette vue sert pour les deux modes du formulaire :

- ajout d'un ordinateur ;
- modification d'un ordinateur existant.

Le mode est déterminé à partir de la route. En modification, les champs sont préremplis avec les données existantes. En ajout, les champs sont initialisés vides.

Le formulaire comprend les champs suivants :

- modèle ;
- processeur ;
- mémoire RAM ;
- disque ;
- carte graphique ;
- photo.

Pour la photo, deux solutions sont disponibles :

1. saisir directement un nom ou un chemin d'image ;
2. cliquer sur « Choisir une image » pour parcourir l'explorateur de fichiers.

Un aperçu de l'image est affiché avant l'enregistrement.

`storage.js`

Ce service centralise la sauvegarde et le chargement des ordinateurs dans `localStorage`. Il permet de retrouver les données après la fermeture ou l'actualisation de la page.

Lors de la première utilisation, cinq ordinateurs d'exemple sont ajoutés automatiquement.

`ordinateurs.js`

Ce fichier contient les données initiales des cinq ordinateurs d'exemple et
les noms des images associées.

`main.js`

Ce fichier initialise l'application Vue et configure les routes :

Route	Fonction
`/ordinateurs`	Afficher la liste
`/ordinateurs/ajouter`	Ajouter un ordinateur
`/ordinateurs/modifier/:id`	Modifier un ordinateur

5. Fonctionnalités réalisées

Consultation

Les ordinateurs sont présentés sous forme de cartes. Chaque carte contient les
informations techniques et la photo correspondante.

Ajout

Le bouton « Ajouter un ordinateur » ouvre le formulaire. Après validation,
le nouvel ordinateur est ajouté à la liste et sauvegardé dans le navigateur.

Modification

Le bouton « Modifier » ouvre le même formulaire avec les données de l'ordinateur
sélectionné déjà renseignées. Les changements sont enregistrés après validation.

Suppression

Le bouton « Supprimer » demande une confirmation avec `confirm()` avant de retirer l'ordinateur. La liste est actualisée immédiatement après la suppression.

Validation

Les contrôles suivants sont appliqués :

- le modèle est obligatoire ;
- le processeur est obligatoire ;
- la mémoire doit être strictement positive ;
- le disque doit être strictement positif ;
- le fichier choisi doit être une image ;
- une photo doit être fournie.

Gestion des images

Les images du projet sont placées dans `src/assets/images/`. Les images livrées avec les exemples sont :

- `Acer Aspire 5.jpeg` ;
- `ASUS TUF Gaming F15.jpeg` ;
- `Dell Latitude 5540.jpeg` ;
- `HP ProBook 450 G10.jpeg` ;
- `Lenovo ThinkPad E14.jpeg`.

Une image choisie depuis l'explorateur est convertie en données intégrées afin de pouvoir être conservée avec l'ordinateur dans `localStorage`.

Recherche et tri

La recherche permet de filtrer les résultats à partir du modèle ou du processeur.

Le tri peut être effectué par :

- modèle ;
- mémoire RAM décroissante ;
- capacité du disque décroissante.

Interface responsive

La grille s'adapte au nombre d'ordinateurs :

- trois cartes sur la première ligne et deux cartes plus larges pour cinq éléments ;
- deux colonnes pour quatre éléments ;
- une seule colonne sur les petits écrans.

Les boutons et les cartes restent lisibles sur ordinateur comme sur mobile.

6. Notions Vue.js mobilisées

Le projet utilise les notions demandées dans le sujet :

- `ref()` pour les valeurs réactives ;
- `computed()` pour les données filtrées et l'aperçu de l'image ;
- `v-for` pour afficher les cartes ;
- `v-if` pour les affichages conditionnels ;
- `v-model` pour les champs du formulaire et les filtres ;
- `@click`, `@submit` et `@change` pour les événements ;
- `props` pour transmettre un ordinateur à une carte ;
- `emit` pour remonter la demande de suppression ;
- Vue Router pour les différentes pages ;
- `localStorage` pour la persistance.

7. Difficultés rencontrées et solutions

Extraction des consignes du sujet PDF

Le sujet était fourni sous forme de PDF et son texte n'était pas directement exploitable comme un fichier texte. Les consignes ont été récupérées afin de respecter précisément les routes, les champs et les fonctionnalités demandées.

Absence initiale de projet Vue

Le dossier contenait uniquement le sujet PDF. Il a donc fallu créer la structure complète du projet Vue 3 avec Vite, les dépendances, les routes, les composants et les vues.

Images non affichées

Les premières données faisaient référence à des noms génériques comme `ordinateur-1.jpg`, alors que les fichiers présents avaient des noms différents et l'extension `.jpeg`.

Le problème a été résolu en associant les données aux vrais noms de fichiers et en rendant le chargement compatible avec plusieurs extensions d'image.

Conservation d'une image choisie par l'utilisateur

Une image sélectionnée dans l'explorateur n'est pas directement importable comme un fichier du projet pendant l'exécution. Elle est donc lue avec `FileReader`, convertie en Data URL et sauvegardée dans `localStorage`.

Adaptation de la grille

Le nombre de cartes pouvant changer après une recherche, une suppression ou un ajout, une grille fixe ne convenait pas toujours. Des classes CSS calculées selon le nombre d'éléments permettent d'obtenir une disposition équilibrée.

Correspondance avec la maquette

L'interface a été ajustée pour reprendre les éléments visuels de la maquette :

- barre supérieure bleue ;
- bouton d'ajout vert ;
- boutons Modifier bleus ;
- boutons Supprimer rouges ;
- cartes compactes avec image en haut ;
- informations techniques alignées sur une ligne.

8. Vérification

La commande suivante a été exécutée avec succès :

```bash
npm run build
```

L'application peut être lancée avec :

```bash
npm run dev
```

9. Conclusion

L'application réalisée répond aux besoins du sujet. Elle permet de gérer efficacement un parc d'ordinateurs dans une interface claire, réutilisable et adaptée aux différentes tailles d'écran. Les données sont conservées localement et l'utilisateur peut également associer une image à chaque équipement.